Project Report Of Computer Vision and AI



Comparative Study of VGG16-BN and ResNet34 Encoders in U-Net for Semantic Segmentation on the CamVid Dataset

Submitted by

Gauri Nitin Wagholikar

Supervised by

Prof. Dr. Tatyana Ivanovska

Ostbayerische Technische Hochschule Amberg-Weiden

Department of Electrical Engineering, Media and Computer Science

June 25, 2026

Abstract

This study presents a controlled comparison of two U-Net variants for semantic segmentation on the CamVid urban driving benchmark, using VGG16-BN and ResNet34 as encoder backbones. Hyperparameters for both architectures were independently optimized using Optuna across 20 trials, searching over learning rate, weight decay, optimizer selection, and the relative weighting of a combined cross-entropy and Dice loss. Both models were subsequently trained for 50 epochs using a cosine annealing learning-rate scheduler.

On the test set, UNet-VGG16-BN achieved a mean intersection-over-union (mIoU) of 0.7123, pixel accuracy of 0.9325, a mean Dice coefficient of 0.8142, and a mean boundary F1 score of 0.6615, with an inference throughput of 112.5 FPS and a computational cost of 76.42 GFLOPs per forward pass. UNet-ResNet34 achieved a mIoU of 0.6729 and a mean boundary F1 score of 0.6079, but operated at 186.2 FPS with only 17.75 GFLOPs, representing a roughly 4.3 times less in computational cost at the expense of a 3.9 point decrease in mIoU. Per-class analysis and qualitative evaluation suggest that differences in skip-connection resolution contribute to the observed segmentation gap, particularly on boundary-heavy and structurally complex classes. Both models show reduced performance on thin and infrequent classes such as ColumnPole and Pedestrian, highlighting the impact of class imbalance in the CamVid dataset.

1 Introduction and Motivation

Semantic segmentation is a basic computer vision task in which a semantic class label is assigned to every pixel in an image. As compared to image classification or object detection, semantic segmentation requires both recognition of object categories and precise localization of object boundaries. This capability is crucial for autonomous driving applications, where an accurate understanding of roads, vehicles, pedestrians, traffic signs, and other scene elements is necessary for safe decision-making.

The challenge in semantic segmentation is balancing semantic understanding with spatial precision. Deep convolutional neural networks generally rely on repeated downsampling operations to learn increasingly abstract feature representations. While downsampling helps the model capture high-level semantic information, it also decreases spatial resolution and can lead to the loss of fine details needed to accurately bound objects.

In order to resolve this issue, U-Net was presented as an encoder-decoder architecture that combines semantic feature extraction with spatial reconstruction. The encoder sequentially shrinks the input representation, while the decoder rebuilds spatial resolution via a series of upsampling operations. An essential part of U-Net is its utilization of skip connections, which forward feature maps from the encoder to matching decoder stages, thereby recovering spatial information that might otherwise be discarded by downsampling.

The overall structure of the decoder is fixed, and different choices of encoder backbones have a significant impact on both segmentation accuracy and computational efficiency. Different encoders produce features with different amounts of spatial detail, receptive fields, and computational requirements. Thus, to study how the encoder influences the U-Net's behavior can help choose an appropriate deployment model.

This study compares two U-Net variants for the CamVid semantic segmentation benchmark. One variant employs a VGG16-BN encoder while the other one uses a ResNet34 encoder. We evaluate not only their accuracy, but also their computational cost, inference latency, and throughput to analyze the tradeoff between segmentation quality and real-time performance.

2 Methodology

The following section describes the full experimental setup used in this study. Covers the data set and its class structure, the U-Net architecture and how each encoder plugs into it, the hyperparameter search procedure carried out with Optuna, and the training configuration including the loss function, optimiser, and learning rate schedule. Each component is explained in enough detail to make the results that follow fully reproducible.

2.1 Overall Architecture

Both models are based on U-Net [1], an encoder-decoder network in which skip connections pass feature maps directly from each encoder stage to the corresponding decoder stage, allowing the decoder to recover spatial detail lost during downsampling. The two configurations share an identical decoder by only the encoder backbone differs, ensuring that any performance gap is attributable to the encoder alone.

Each DecoderBlock upsamples via bilinear interpolation (`align_corners=False`), concatenates the corresponding encoder skip, and applies two Conv-BN-ReLU layers. A final bilinear interpolation restores the output to 256×256, and a 1×1 convolution maps features to 11 class scores per pixel.

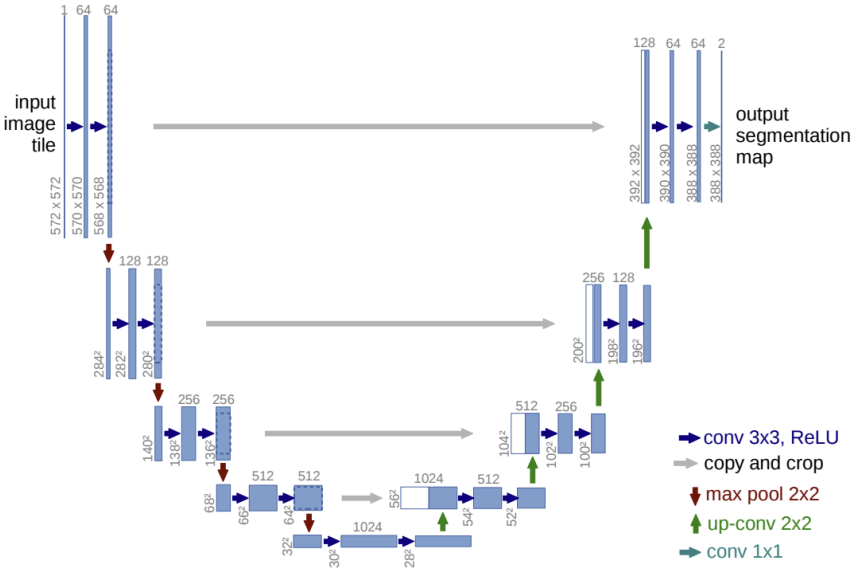


Figure 1: U-Net encoder-decoder architecture with skip connections.

2.2 Encoder Backbones

Table 1: VGG16-BN encoder stages for a 256×256 input.

Stage	Components	Resolution	Channels
1	Conv-BN-ReLU ×2	256 × 256	64
2	Conv-BN-ReLU ×2	128 × 128	128
3	Conv-BN-ReLU ×3	64 × 64	256
4	Conv-BN-ReLU ×3	32 × 32	512
5	Conv-BN-ReLU ×3 (bottleneck)	16 × 16	512

Table 2: ResNet34 encoder skip levels for a 256×256 input.

Level	Components	Resolution	Channels
0	conv1 + BN + ReLU	128 × 128	64
1	MaxPool + layer1 (residual ×3)	64 × 64	64
2	layer2 (residual ×4)	32 × 32	128
3	layer3 (residual ×6)	16 × 16	256
4	layer4 (residual ×3, bottleneck)	8 × 8	512

2.3 VGG16-BN:

The VGG16 encoder with Batch Normalisation [2] [3] is split into five stages at each MaxPool boundary, using pretrained ImageNet weights. Stage 1 operates at full input resolution before any downsampling occurs, providing the decoder with a full-resolution skip connection. Table 1 summarises the five stages.

2.4 ResNet34:

The ResNet34 encoder [4] uses residual connections to address vanishing gradients in deep networks, and is also initialised with pretrained ImageNet weights. Unlike VGG16-BN, the stem (conv1 + BN + ReLU) immediately downsamples by 2×, and a subsequent MaxPool adds a further 2× before the first residual block. Consequently, the shallowest skip is at 128×128 rather than 256×256, and the first residual layer's skip is already at 64×64. Table 2 summarises the five skip levels.

Comparing Tables 1 and 2, the key difference is that ResNet34 discards fine spatial detail before any skip information reaches the decoder. This architectural difference may influence the amount of spatial information available to the decoder and is therefore considered a potential factor affecting segmentation quality.

2.5 Loss Function

Training minimises a weighted combination of Cross-Entropy (CE) and Dice loss:

$$\mathcal{L} \;=\; w_{\mathrm{CE}}\,\mathcal{L}_{\mathrm{CE}} \;+\; w_{\mathrm{Dice}}\,\mathcal{L}_{\mathrm{Dice}}, \qquad w_{\mathrm{CE}} + w_{\mathrm{Dice}} = 1. \tag{1}$$

CE loss penalises each pixel independently, while Dice loss optimises class overlap directly, providing robustness to the severe pixel-frequency imbalance present in CamVid [5] (e.g. *Road* accounts for 31.9% of training pixels). The weights w_{CE} and w_{Dice} were determined automatically by hyperparameter optimisation (Section 2.6).

2.6 Hyperparameter Optimisation

Hyperparameters were selected using Optuna [6], which uses Tree-structured Parzen Estimation (TPE) to focus sampling on promising regions of the search space. Twenty trials were run per encoder, each training for 20 epochs and maximising validation mIoU. A `MedianPruner` with two warm-up steps terminated underperforming trials early. The search space covered learning rate (log-uniform, $[10^{-5}, 10^{-3}]$), batch size ($\{2, 4\}$), optimiser (Adam / AdamW), weight decay (log-uniform, $[10^{-6}, 10^{-2}]$), and CE loss weight (uniform, $[0.2, 0.8]$). The best configuration for each encoder is reported in Table 3.

Table 3: Training configurations and best validation results for both runs.

Setting	**UNet-VGG16**	**UNet-ResNet34**
Best val mIoU	0.748	0.720
CE weight	0.623	0.557
Dice weight	0.377	0.443
Learning rate	1.10×10^{-4}	1.84×10^{-4}
Batch size	2	2
Optimizer	AdamW	Adam
Weight decay	5.43×10^{-5}	3.19×10^{-6}

Both models converged to batch size 2. VGG16-BN preferred AdamW (decoupled weight decay), while ResNet34 preferred Adam with a much smaller weight decay. Optuna selected a higher Dice-loss weight for ResNet34 than for VGG16. This suggests that stronger region-based supervision may have been beneficial for this architecture under the evaluated training configuration.

2.7 Training Setup

Both models were trained for 50 epochs using the hyperparameters in Table 3. The learning rate followed cosine annealing, decaying smoothly to near zero by the final epoch. Gradient clipping at a maximum ℓ_2 norm of 1.0 was applied throughout, and Automatic Mixed Precision (AMP) was enabled on CUDA to reduce memory usage. Early stopping with patience 10 was implemented but not triggered within the 50-epoch budget. A global seed of 42 was fixed for reproducibility. Augmentation (horizontal flips, colour jitter, random crops) was applied to the training set only. Validation and test sets used resizing and normalisation exclusively. The full configuration is given in Table 4. Training progress was monitored using validation mIoU, and model checkpoints were saved whenever validation performance improved.

Table 4: Training configuration shared by both models.

Setting	**Value**
Image size	256×256
Training epochs	50
Batch size	2
LR scheduler	Cosine Annealing
Gradient clipping	1.0 (max ℓ_2 norm)
Mixed precision (AMP)	Enabled (CUDA)
Early stopping patience	10 epochs
Random seed	42
Augmentation	Training set only

3 Results and Discussion

3.1 Quantitative Results

Tables 5 and 6 show the segmentation quality and efficiency numbers for both models on the held-out CamVid test set.

Table 5: Segmentation quality metrics on the CamVid test set.

Metric	UNet-VGG16	UNet-ResNet34
mIoU	**0.7123**	0.6729
Pixel Accuracy	**0.933**	0.923
Mean Dice	**0.814**	0.781
Macro Precision	**0.844**	0.811
Macro Recall	**0.791**	0.760
Mean Boundary F1	**0.662**	0.608

Table 6: Computational efficiency metrics on the CamVid test set.

Metric	UNet-VGG16	UNet-ResNet34
Parameters	25.79M	31.57M
Model Size	98.4 MB	120.4 MB
GFLOPs	76.42	**17.75**
GPU Peak Memory	199.3 MB	**178.6 MB**
Avg. Latency	8.89 ms	**5.37 ms**
FPS	112.5	**186.2**

To ensure a fair comparison, both models were trained using identical data splits, augmentation strategies, evaluation metrics, and training schedules, with the encoder backbone being the primary architectural difference. VGG16-BN wins on every segmentation quality metric and ResNet34 wins on every efficiency metric. VGG16-BN pulls ahead by 3.9 mIoU points (0.712 vs 0.673), and the boundary F1 gap is around 5.4 points (0.662 vs 0.608). That boundary F1 difference is probably the more telling number here, because it measures how cleanly a model captures the actual edges between classes rather than just getting the broad region roughly right. The per-class boundary F1 plot confirms this pattern across almost every category, with the largest gaps appearing on ColumnPole (0.52 vs 0.42) and Fence (0.53 vs 0.44), which are exactly the kinds of thin and structurally complex classes where fine spatial detail matters most.

On the efficiency side, ResNet34 runs at 186.2 FPS compared to VGG16-BN's 112.5 FPS, and its average latency is noticeably lower (5.37 ms vs 8.89 ms). The inference time distribution plot over 50 runs confirms this gap is consistent and not just a one-off measurement. This is mostly explained by the GFLOPs difference, which is not marginal at all. VGG16-BN needs 76.42 GFLOPs per forward pass and ResNet34 only needs 17.75, so roughly a 4.3x gap. And there is also a slightly counterintuitive finding here as ResNet34 actually has more parameters (31.57M vs 25.79M) and a larger file on disk (120.4 MB vs 98.4 MB), yet it uses less GPU memory at inference (178.6 MB vs 199.3 MB). The parameters and GPU memory plot makes this contrast easy to see. This happens because inference memory is dominated by the intermediate activation maps created during the forward pass, not by the stored weights. VGG16-BN keeps large full-resolution feature maps alive through its skip connections, and those eat up considerably more memory than ResNet34's more aggressively downsampled activations. So parameter count alone is a pretty poor proxy for how expensive a model actually is to run.

3.2 Per-Class Analysis

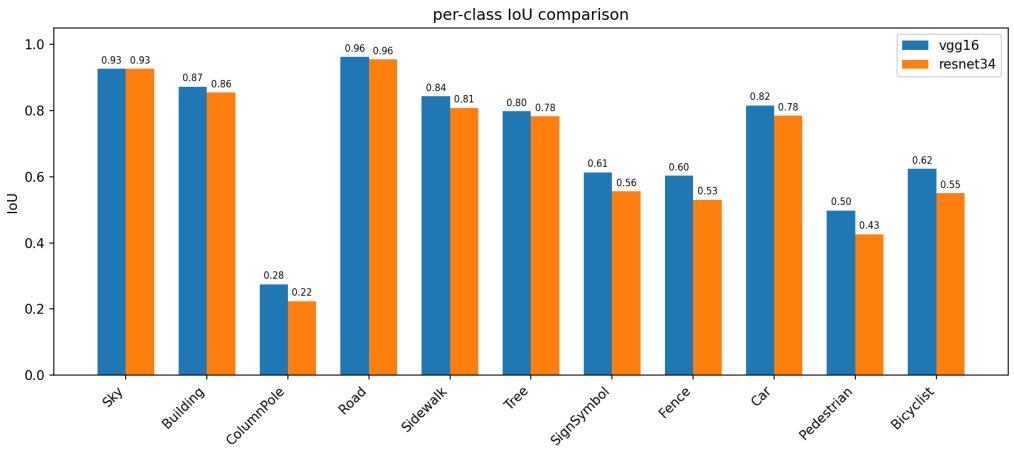


Figure 2: Per-class IoU comparison between UNet-VGG16-BN and UNet-ResNet34 on the CamVid test set. VGG16-BN leads on most categories, with the largest gaps on structurally complex and boundary-heavy classes. Both models struggle on thin, infrequent categories such as ColumnPole and Pedestrian.

Figure 2 breaks the mIoU down by class and the pattern becomes a lot clearer. Both models handle the big, texturally simple classes like Sky (0.93 each) and Road (0.96 each) quite well. Those are the easy ones, and the gaps between the two models are essentially zero there. The class distribution plot also helps explain why. Road accounts for 31.9% of training pixels and Building for 25%, so the model gets an enormous amount of signal for those categories. Where VGG16-BN really pulls ahead is on the harder categories like Fence (0.60 vs 0.53), Bicyclist (0.62 vs 0.55), and Pedestrian (0.50 vs 0.43). These are the classes where getting the boundary right actually matters, and this is probably where that full-resolution skip connection in VGG16-BN is doing the most work, giving the decoder access to fine spatial detail that ResNet34's decoder simply does not have at the same resolution.

The weakest classes for both models are the thin, infrequent ones. ColumnPole is the worst for both (0.28 vs 0.22), and Pedestrian is not far behind. Neither model does well there, and honestly that is not too surprising. At 256x256 resolution these objects are only a handful of pixels wide, and the class distribution plot shows that ColumnPole represents just 1.1% of training pixels. The model barely gets enough signal to learn them properly. This is more of a class imbalance problem in CamVid than an architectural one.

3.3 Training Dynamics

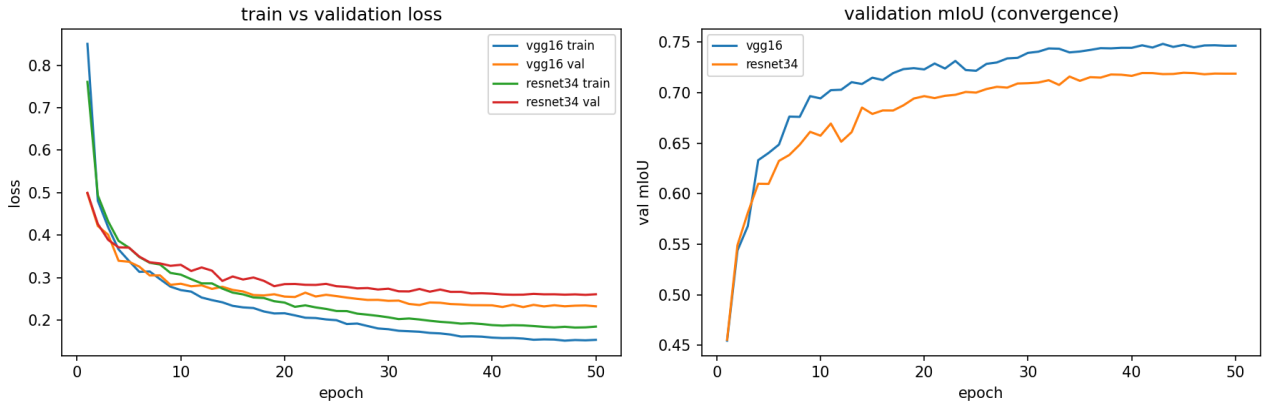


Figure 3: Training and validation loss curves (left) and validation mIoU convergence over 50 epochs (right) for both encoders. VGG16-BN converges steadily and continues improving into the final epochs. ResNet34 converges at a similar pace but plateaus earlier and at a lower final mIoU.

In Figure 3, the two models show quite different training dynamics. VGG16-BN started from a mIoU of 0.4545 in epoch 1 and climbed steadily the whole way through, crossing 0.70 by around epoch 11 and continuing to improve, with its best checkpoint landing at 0.7481 in epoch 43. The loss curves are smooth and the gap between training and validation loss stays healthy throughout, which suggests the model was learning genuinely useful features rather than just memorising the training set.

ResNet34 also got off to a reasonable start, crossing 0.60 by epoch 4 and hitting 0.70 by around epoch 24. But from around epoch 30 onward the validation mIoU stopped improving in any meaningful way, hovering between 0.71 and 0.72 for the rest of training and finishing with a best checkpoint of 0.7196 at epoch 45. The overfitting gap plot shows that both models maintain a healthy train-validation gap throughout, so this is not really an overfitting issue. It looks more like ResNet34 simply reached its ceiling earlier under this training configuration. The loss curve stability plot shows that both models have very similar instability patterns in the early epochs, with rolling standard deviation spiking around epochs 3 to 5 before settling down. After that both are quite stable, so the plateau in ResNet34 is probably not a training instability problem either.

The validation pixel accuracy plot tells a similar story. VGG16-BN climbs from around 0.87 at epoch 1 to around 0.946 by the end, while ResNet34 reaches around 0.940, a gap that is consistent with the mIoU difference. Both curves are smooth and monotonically increasing, which is reassuring.

Each VGG16-BN epoch took around 7.1 seconds compared to ResNet34's 5.9 seconds on average, as seen in the training time per epoch plot. The learning rate schedule plot shows that ResNet34 started with a higher initial learning rate (1.84e-4 vs 1.10e-4) and decayed along the same cosine curve shape, both reaching near zero by epoch 50. The gradient norm distribution plot shows that both models have very similar gradient behaviour across training, with the vast majority of norms concentrated below 2, which confirms that gradient clipping was active throughout. However, inspection of the raw gradient norm logs reveals two isolated steps per model that produced an infinite gradient norm, likely caused by AMP fp16 overflow events. These occurred near epochs 28 and 50 for VGG16, and near epochs 23 and 45 for ResNet34. They did not visibly disrupt convergence in either case, and the remaining 9,198 steps per model show the stable, sub-2 pattern visible in the distribution

plot. This kind of isolated overflow is common with mixed-precision training and is worth noting rather than ignoring.

Optuna picked a higher Dice-loss weight for ResNet34 (0.443) than for VGG16-BN (0.377). That seems to indicate that stronger region-based supervision helped out ResNet34 during optimisation. But we do not look into the reason behind this difference here and more experiments would be necessary to understand why a higher Dice weighting was preferred.

3.4 Efficiency Trade-off

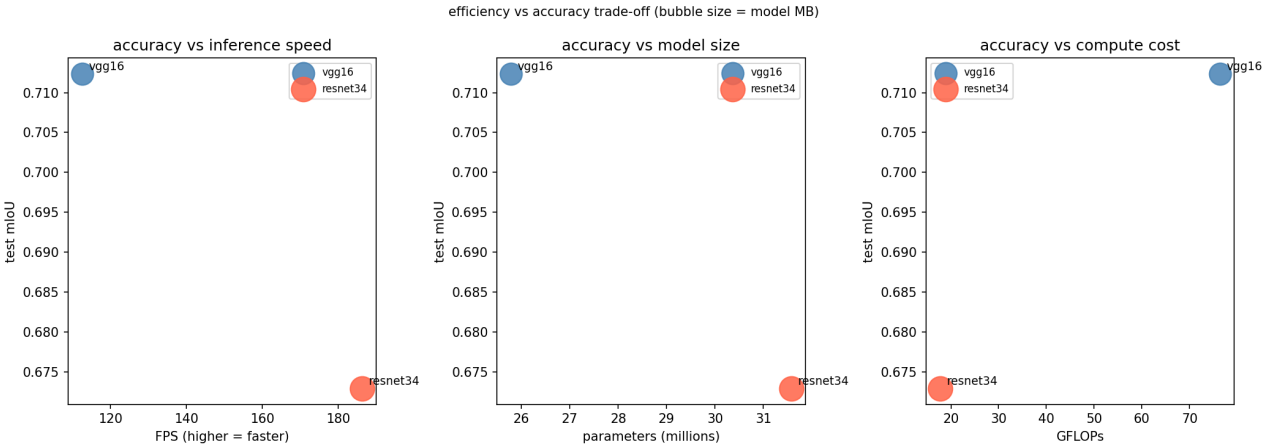


Figure 4: Accuracy vs efficiency trade-off across three dimensions. Left: mIoU vs FPS. Centre: mIoU vs parameter count. Right: mIoU vs GFLOPs. Bubble size encodes model size in MB. ResNet34 is substantially faster and cheaper to run, at the cost of a 3.9-point mIoU gap.

Figure 4 puts the accuracy-efficiency trade-off into a visual context and it makes the deployment choice quite concrete. VGG16-BN is the better model for segmentation quality, and at 112.5 FPS it also clears the 60 FPS threshold that is commonly expected for real-time applications. ResNet34 at 186.2 FPS clears that bar even more comfortably. So in this experiment both models are technically capable of real-time inference, but ResNet34 does it at roughly 4.3x lower compute cost. The accuracy vs model size plot is also interesting because it shows that ResNet34 has more parameters and a larger file on disk, yet still runs faster and cheaper. That is a good reminder that parameter count is not a reliable indicator of runtime cost. The question of which model to use is really a question of what the application actually needs. If boundary precision matters and compute budget is flexible, VGG16-BN is the right choice. If the system needs to run fast on constrained hardware with minimal GFLOPs, ResNet34 is probably the more sensible option despite the accuracy gap.

3.5 Qualitative Analysis

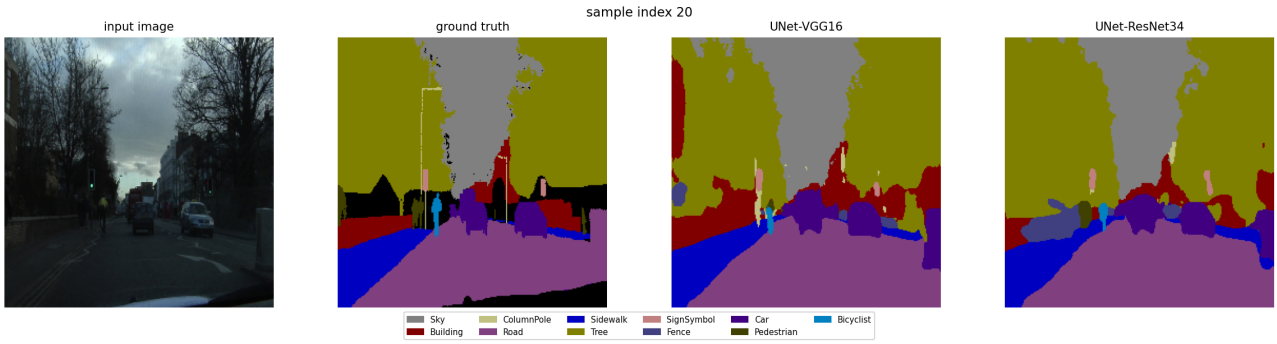


Figure 5: Prediction comparison on a representative urban driving scene (sample index 20). From left to right: input image, ground truth, UNet-VGG16-BN prediction, UNet-ResNet34 prediction. VGG16-BN produces cleaner boundaries around the tree canopy and building edges. ResNet34 correctly identifies the broad regions but shows class bleeding at finer boundaries.

Figure 5 shows a fairly typical urban driving scene with road, buildings, trees, and sky. Both models get the big regions right, which is reassuring. But the difference shows up at the edges. VGG16-BN draws noticeably cleaner lines around the tree canopy and at the transition between buildings and sky. ResNet34's prediction is more blob-like in those areas, with class regions bleeding into each other where the boundary is thin or ambiguous. This is not a dramatic failure but it is a consistent pattern, and it maps directly onto the boundary F1 gap from Table 5.

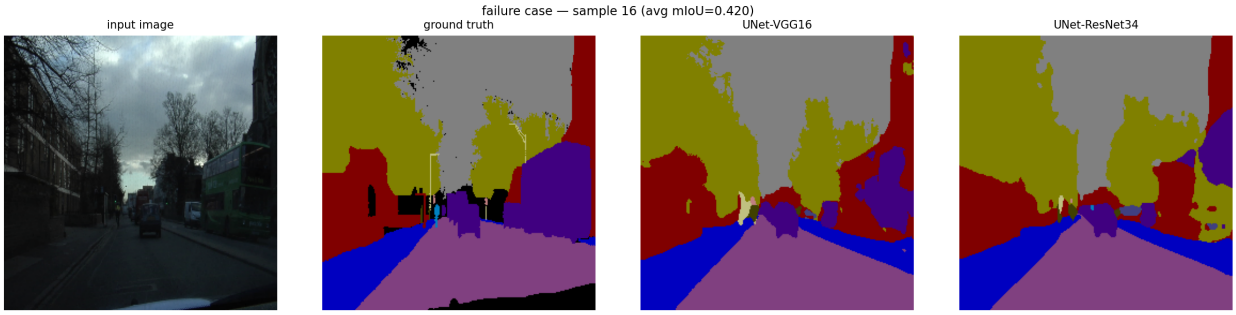


Figure 6: Failure case: sample index 16 (avg mIoU = 0.420, VGG16-BN = 0.437, ResNet34 = 0.403). Dark lighting and cluttered thin structures cause both models to produce coarse and inaccurate segmentation maps. Neither model recovers thin vertical structures such as lamp posts and sign poles.

Figure 6 is one of the hardest samples in the test set, with an average mIoU of just 0.420 across both models. The scene is dark, cluttered, and full of thin vertical structures like lamp posts and poles. Neither model handles it well at all. The scores are close (VGG16-BN at 0.437, ResNet34 at 0.403), and what is interesting is that VGG16-BN does hold a small edge here too, which is consistent with its generally better boundary handling. But both models are clearly failing for the same underlying reason, which is that these thin structures are nearly invisible at 256x256 resolution and represent so few training pixels that the model has almost nothing to learn from. The predictions collapse those pixels into neighbouring dominant classes, and no amount of skip connection resolution is going to fix a class imbalance problem that severe.

3.6 Confusion Matrix Analysis

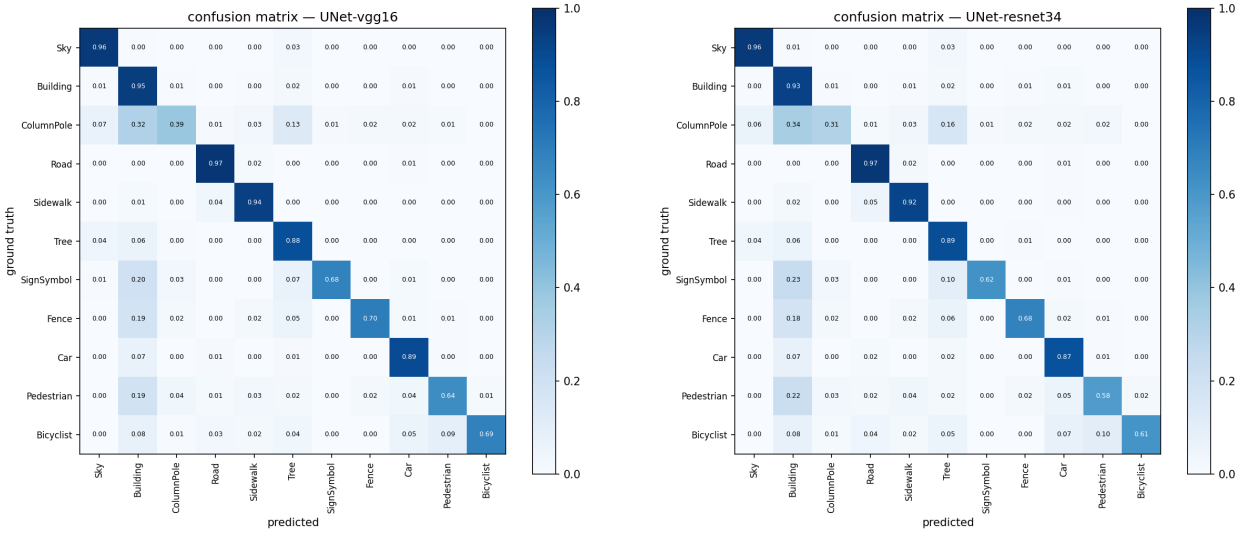


Figure 7: Normalised confusion matrices for UNet-VGG16-BN (left) and UNet-ResNet34 (right). Both models show strong diagonal dominance on large classes. The most frequent off-diagonal confusions occur between visually similar categories such as Buildings and ColumnPoles, and between Pedestrians and Bicyclists.

The confusion matrices in Figure 7 show that both models have strong diagonal dominance, meaning they are getting the right class the majority of the time for most categories. Sky scores 0.96 on the diagonal for both models, and Road scores 0.97, which is slightly higher than its IoU of 0.96 because the diagonal here reflects per-class recall rather than the intersection-over-union metric. The off-diagonal confusions that do show up are not random. ColumnPole is the most confused class in both matrices, with a diagonal score of only 0.39

for VGG16-BN and 0.31 for ResNet34. A large fraction of ColumnPole pixels get absorbed into Building (0.32 and 0.34 respectively), which makes sense given that poles often appear directly in front of building facades and share similar colour tones. Pedestrian is also notably confused, with both models sending a chunk of pedestrian pixels to Building (0.19 for VGG16-BN and 0.22 for ResNet34). VGG16-BN scores slightly better on Pedestrian (0.64 diagonal vs 0.58) and Bicyclist (0.69 vs 0.61), which is consistent with its higher recall and precision numbers overall. SignSymbol is another weak class for both, with ResNet34 at 0.62 and VGG16-BN at 0.68, and a notable fraction being confused with Building and Tree. These are all visually understandable confusions. VGG16-BN's matrix is slightly cleaner on these harder pairs, which lines up with its higher overall metrics.

3.7 Limitations

1. The models are trained and evaluated only on CamVid, which is a relatively small dataset, so the results are achieved and applicable only for the CamVid dataset.

2. Both models are trained at a fixed input resolution of 256x256, which suppresses fine boundary detail that would be visible at higher resolutions.

3. The class imbalance present in CamVid, where large regions like Road and Building dominate, is only partially addressed by the combined loss, and rare classes such as ColumnPole and Pedestrian remain noticeably harder for both models.

4. The hyperparameter search is limited to 20 Optuna trials per encoder, which may not be enough to explore the search space fully and could mean the found configurations are not globally optimal [6].

3.8 Future Work

1. To study the same VGG16-BN and ResNet34 encoders within a SegNet architecture and compare whether the asymmetric encoder-decoder design of SegNet handles boundary reconstruction differently from U-Net on the same CamVid benchmark.

2. Observe and study the results on a bigger dataset with higher resolution.

4 Conclusion

The central finding of this work is that when spatial detail and boundary precision matter, UNet-VGG16-BN is the stronger choice. When the system needs to run fast and cheaply, UNet-ResNet34 is the one to use. And the reason the two models end up in such different places comes down to one architectural decision, which is where the shallowest skip connection sits.

VGG16-BN's encoder produces a full-resolution skip at 256x256, meaning the decoder gets access to fine spatial detail from the very first reconstruction step. This is why VGG16-BN not only scores higher on mIoU (0.7123 vs 0.6729) but also leads on boundary F1 (0.6615 vs 0.6079), which is arguably the harder and more practically relevant part of the segmentation task. Getting broad regions roughly right is one thing. Drawing clean, accurate boundaries around pedestrians, fences, and bicyclists in a real driving scene is another. VGG16-BN does both. ResNet34 does the first one well but struggles with the second, because its stem already downsamples the input before any residual block runs, and that loses half the spatial resolution, which is very hard to recover later in the decoder no matter how the loss is weighted.

However, ResNet34 is not a weak model, it is a trade-off between efficiency and accuracy. Running at 186.2 FPS with 5.37 ms latency and only 17.75 GFLOPs per forward pass, it is a genuinely practical model for real-time deployment. The accuracy gap of 3.9 mIoU points is real, but it is not catastrophic, and for many applications that trade-off is worth making. Both models also cleared the 60 FPS real-time threshold, so the choice between them is more about how much accuracy the application can afford to give up rather than whether real-time inference is possible at all.

This comparison also reveals that neither model solves the harder problem in this dataset, which is the thin, infrequent structural classes like ColumnPole and Pedestrian. Both models collapse on those, and the confusion matrix analysis shows that ColumnPole pixels frequently get absorbed into Building for both architectures. The failure case analysis also shows that on the hardest samples both models score similarly (VGG16-BN at 0.437, ResNet34 at 0.403). That tells us the remaining headroom is not purely an encoder problem. It is also a data problem, and closing it would probably require better handling of class imbalance, higher input resolution, or both.

Declaration

Gemini [7] was used for assisting in coding and debugging of this project.

References

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, ser. Lecture Notes in Computer Science, vol. 9351, Springer, Cham, 2015, pp. 234–241. [Online]. Available: `https://arxiv.org/abs/1505.04597`.

[2] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2015. [Online]. Available: `https://arxiv.org/abs/1409.1556`.

[3] V. Badrinarayanan, A. Kendall, and R. Cipolla, "Segnet: A deep convolutional encoder-decoder architecture for image segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 12, pp. 2481–2495, 2017. [Online]. Available: `https://arxiv.org/pdf/1511.00561`.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: `https://arxiv.org/abs/1512.03385`.

[5] G. J. Brostow, J. Shotton, J. Fauqueur, and R. Cipolla, *CamVid – cambridge-driving labelled video database*, `https://mi.eng.cam.ac.uk/research/projects/VideoRec/CamVid/`, 2008.

[6] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," *arXiv preprint arXiv:1907.10902*, 2019. [Online]. Available: `https://arxiv.org/abs/1907.10902`.

[7] Google, *Gemini*, `https://gemini.google.com/`, Large language model developed by Google. This specific interaction occurred on July 2, 2025. Accessed via Google, 2025. [Online]. Available: `https://gemini.google.com/`.